

Integridade de Dados com Constraints no MySQL

Estudo de caso aplicado ao sistema de plano de saúde

(Use: PlanoSaudeDB.sql)

 **por Salvador Melo**





Por que aprender constraints?



Segurança clínica

Erros de dados podem comprometer diagnósticos, cobranças e atendimentos.



Coerência garantida

Constraints ajudam a garantir informações coerentes, seguras e válidas.



Prevenção de erros

Impedem que um CPF seja cadastrado duas vezes ou valores impossíveis.

O que são constraints?

Regras declarativas

São aplicadas diretamente nas colunas ou tabelas do banco de dados.

Prevenção automática

Evitam dados incorretos desde o momento da inserção.

Execução pelo SGBD

São executadas automaticamente pelo banco de dados.



Tipos de constraints no MySQL



PRIMARY KEY

FOREIGN KEY

Integridade entre tabelas



NOT NULL

Campo obrigatório



ENUM

Integridade entre tabelas



CHECK

Regra condicional



UNIQUE

Valor exclusivo



CASCADE

on delete

on update



DEFAULT

valor padrão

NOT NULL — Campos obrigatórios

Definição

Exige que o campo tenha valor ao inserir ou atualizar dados.

Em saúde, dados faltantes podem comprometer triagens e diagnósticos.

Exemplo em SQL

```
nome VARCHAR(100) NOT NULL,  
data_nascimento DATE NOT NULL
```

Domínio e segurança no status de pagamento



Problema

A ausência de ENUM em campos como status de pagamento pode parecer inofensiva num banco pequeno, mas quando o sistema cresce, os problemas se multiplicam.



Solução

ENUM assegura padronização dos dados.



Benefício

Relatórios precisos e filtros confiáveis.

Sem ENUM, qualquer pessoa (ou sistema) pode inserir:

- 'pago'
- 'Pago'
- 'PAGO'
- 'pago parcial'
- 'pago_parcial'
- 'em aberto'
- 'Aguardando Pagamento'
- 'Aguardando'
- 'Aguarndando' (erro de digitação)

Restrições de Domínio com ENUM

Conceito

Usamos ENUM quando o campo deve aceitar apenas valores específicos predefinidos.

Impede erros como 'feminino' ou 'Pleno' quando esperamos formatos padronizados.

Exemplos práticos

```
CREATE TABLE Beneficiarios (  
  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  
  nome VARCHAR(100),  
  
  data_nascimento DATE,  
  
  sexo ENUM('M', 'F', 'Outro'),  
  
  cpf CHAR(11) UNIQUE,  
  
  status ENUM('Ativo', 'Inativo', 'Cancelado'),  
  
  plano_id INT,  
  
  data_adesao DATE );
```

UNIQUE — Garantia de unicidade



Identificação única
Evita duplicações de informações críticas.



Implementação
CREATE TABLE Beneficiarios (
id INT AUTO_INCREMENT PRIMARY KEY,
nome VARCHAR(100),
data_nascimento DATE,
sexo ENUM('M', 'F', 'Outro'),
cpf CHAR(11) UNIQUE,
status ENUM('Ativo', 'Inativo', 'Cancelado'),
plano_id INT,
data_adesao DATE);



Resultado
CPF não pode estar associados a múltiplos registros.

Exemplo prático com UNIQUE



Cadastro inicial

```
INSERT INTO Beneficiarios (nome, data_nascimento, sexo, cpf, status, plano_id, data_adesao)
VALUES ('Anthony Oliveira', '2013-07-27', 'M', '92065478101', 'Cancelado', 5, '2021-01-31'),
```



Tentativa de duplicação

```
INSERT INTO Beneficiarios (nome, data_nascimento, sexo, cpf, status, plano_id, data_adesao)
VALUES ('Stephany Vieira', '1974-03-13', 'F', '92065478101', 'Cancelado', 10, '2020-10-03'),
```



Bloqueio automático

Error Code: 1062. Duplicate entry '92065478101' for key 'beneficiarios.cpf' 0.047 sec

4

Benefício

Evita registros duplicados e conflitos de agendamento ou cobrança.



CHECK — Regras condicionais

Para garantir que a data de nascimento de um beneficiário seja menor ou igual à data de hoje no momento da criação da tabela, você pode usar uma restrição **CHECK** diretamente na definição da tabela.

```
CREATE TABLE Beneficiarios (  
  
id INT AUTO_INCREMENT PRIMARY KEY,  
  
nome VARCHAR(100),  
  
data_nascimento DATE,  
  
sexo ENUM('M', 'F', 'Outro'),  
  
cpf CHAR(11) UNIQUE,  
  
status ENUM('Ativo', 'Inativo', 'Cancelado'),  
  
plano_id INT,  
  
data_adesao DATE );
```

```
CREATE TABLE Beneficiarios (  
  
id INT AUTO_INCREMENT PRIMARY KEY,  
  
nome VARCHAR(100),  
  
data_nascimento DATE CHECK (data_nascimento <= CURDATE()),  
  
sexo ENUM('M', 'F', 'Outro'),  
  
cpf CHAR(11) UNIQUE,  
  
status ENUM('Ativo', 'Inativo', 'Cancelado'),  
  
plano_id INT,  
  
data_adesao DATE );
```

CHECK — Quando usar?



Validar faixas permitidas

Ideal para idade, porcentagens e limites numéricos.



Combinações simples

Verificar relações entre campos da mesma tabela.



Validações temporais

Impedir consultas marcadas para datas passadas.

Listar todas as constraints existentes em um banco de dados MySQL

Para listar todas as **constraints** existentes em um banco de dados MySQL, você pode consultar as **tabelas internas do dicionário de dados** (`INFORMATION_SCHEMA`), que armazenam metadados sobre o banco.

```
SELECT
    CONSTRAINT_NAME,
    TABLE_NAME,
    CONSTRAINT_TYPE
FROM
    INFORMATION_SCHEMA.TABLE_CONSTRAINTS
WHERE CONSTRAINT_SCHEMA = 'PlanoSaudeDB';
```

CONSTRAINT_NAME	TABLE_NAME	CONSTRAINT_TYPE
PRIMARY	auditoriaprocedimentos	PRIMARY KEY
auditoriaprocedimentos_ibfk_1	auditoriaprocedimentos	FOREIGN KEY
PRIMARY	autorizacoes	PRIMARY KEY
autorizacoes_ibfk_1	autorizacoes	FOREIGN KEY
autorizacoes_ibfk_2	autorizacoes	FOREIGN KEY
autorizacoes_ibfk_3	autorizacoes	FOREIGN KEY
PRIMARY	beneficiarios	PRIMARY KEY
cpf	beneficiarios	UNIQUE
PRIMARY	consultas	PRIMARY KEY
consultas_ibfk_1	consultas	FOREIGN KEY
consultas_ibfk_2	consultas	FOREIGN KEY
consultas_ibfk_3	consultas	FOREIGN KEY
PRIMARY	planos	PRIMARY KEY
PRIMARY	prestadores	PRIMARY KEY
PRIMARY	procedimentos	PRIMARY KEY
codigo_tuss	procedimentos	UNIQUE
PRIMARY	reembolsos	PRIMARY KEY
reembolsos_ibfk_1	reembolsos	FOREIGN KEY
reembolsos_ibfk_2	reembolsos	FOREIGN KEY

CONSTRAINT_NAME	TABLE_NAME	COLUMN_NAME	REFERENCED_TABLE_NAME	REFERENCED_COLUMN_NAME
auditoriaprocedimentos_ibfk_1	auditoriaprocedimentos	consulta_id	consultas	id
autorizacoes_ibfk_1	autorizacoes	beneficiario_id	beneficiarios	id
autorizacoes_ibfk_2	autorizacoes	prestador_id	prestadores	id
autorizacoes_ibfk_3	autorizacoes	procedimento_id	procedimentos	id
consultas_ibfk_1	consultas	beneficiario_id	beneficiarios	id
consultas_ibfk_2	consultas	prestador_id	prestadores	id
consultas_ibfk_3	consultas	procedimento_id	procedimentos	id
reembolsos_ibfk_1	reembolsos	beneficiario_id	beneficiarios	id
reembolsos_ibfk_2	reembolsos	procedimento_id	procedimentos	id

Ver os detalhes das FOREIGN KEYS

```
SELECT
    CONSTRAINT_NAME,
    TABLE_NAME,
    COLUMN_NAME,
    REFERENCED_TABLE_NAME,
    REFERENCED_COLUMN_NAME
FROM
    INFORMATION_SCHEMA.KEY_COLUMN_USAGE
WHERE
    CONSTRAINT_SCHEMA = 'PlanoSaudeDB'
AND
    REFERENCED_TABLE_NAME IS NOT NULL;
```



Comparação prática das constraints

Situação	Constraint ideal
Garantir CPF único	UNIQUE
Sexo só M, F ou Outro	ENUM
Validar coparticipação	CHECK
Consultas ligadas a pacientes	FOREIGN KEY
Impedir campos vazios	NOT NULL

Validação no banco vs. na aplicação

Banco de dados

- Regra sempre será aplicada
- Independe da aplicação
- Mais seguro e confiável
- Exemplo: NOT NULL, UNIQUE

Aplicação

- Depende do código
- Pode ter falhas ou brechas
- Mais flexível e personalizada
- Mais sujeita a erros

Restrições e desempenho

0.001s

Tempo médio

Constraints nativas do banco são extremamente rápidas.

100%

Confiabilidade

Garantia total de aplicação das regras.

0

Retrabalho

Evitam duplicação de lógica na aplicação.



Boas práticas

1 Segurança máxima

Comece sempre usando o máximo de constraints possíveis.



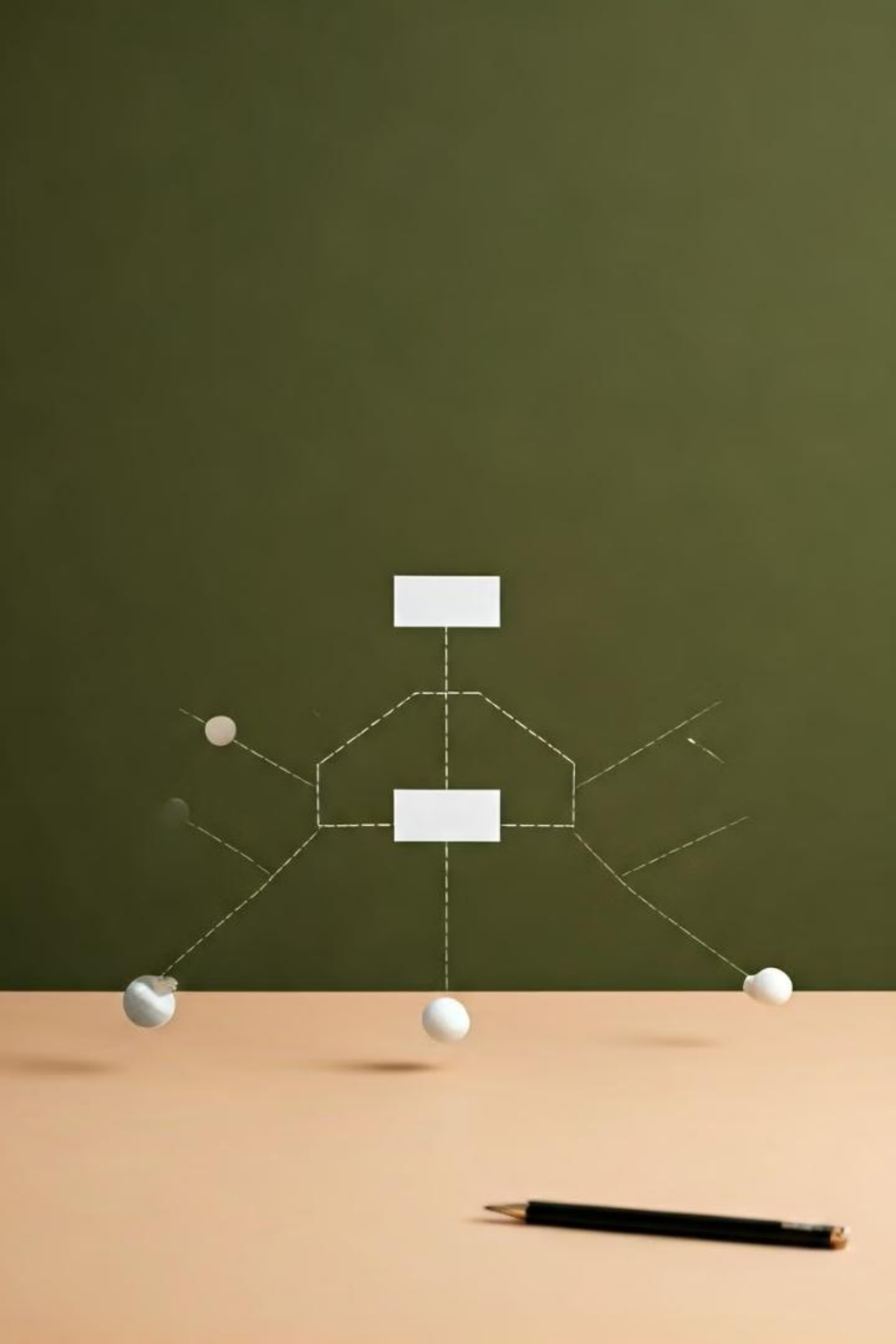
ENUMs com cautela

Use ENUMs com cuidado, pois alterar pode ser trabalhoso.



Documentação

Documente todas as regras aplicadas no banco de dados.





O que o MySQL ainda não faz bem

Compatibilidade

CHECK pode ser ignorado em versões antigas (< 8.0).

Validações complexas

Não permite validações com JOIN ou múltiplas tabelas.

Alternativa

Use triggers para regras mais sofisticadas.

Nenhuma das restrições acima permite validações complexas com **JOIN** ou múltiplas tabelas no MySQL. Elas garantem a integridade e as regras de negócios dentro da própria tabela ou entre tabelas relacionadas (como é o caso de **FOREIGN KEY**), mas todas elas têm um alcance limitado. Para validações mais complexas que envolvem múltiplas tabelas, seria necessário usar **triggers** ou **procedures**, ou até mesmo realizar as validações no lado da aplicação.

Comparando: Constraints no banco de dados vs Validação na aplicação

Critério	Constraints no Banco	Validação na Aplicação
Onde ocorre	Dentro do SGBD (MySQL)	No código (PHP, Java, Python)
Execução	Automática pelo banco	Depende do programador
Segurança	Alta — regras obrigatórias	Média — pode ser burlada
Mensagens	Técnicas (Duplicate entry)	Pode ser personalizada
Manutenção	Centralizada no banco	Espalhada pelo código
Exemplo	cpf UNIQUE, CHECK (idade > 0)	if (idade > 0) { salvar(); }
Risco	Muito baixo	Alto se validação for esquecida

 **No PlanoSaudeDB:** O banco com constraints (**cpf VARCHAR(14) NOT NULL UNIQUE**) nunca permite CPFs duplicados. Já a validação apenas no app pode ser burlada por acessos diretos ao banco.

 **Conclusão prática:** Use constraints sempre que possível. A validação na aplicação é complementar — não substitui as regras de integridade no banco.

ON DELETE CASCADE — Exclusão em cadeia

ON UPDATE CASCADE — Atualização em cadeia

- ♦ **Tecnicamente**, **CASCADE** não é uma **constraint**.
- ♦ Mas... **faz parte do comportamento associado a uma constraint**, mais especificamente à **FOREIGN KEY**.

O **CASCADE** pode ser usado tanto para **exclusão** quanto para **atualização**. Se um beneficiário for excluído, todas as consultas associadas a ele também serão excluídas. Se o identificador do beneficiário for atualizado, o identificador nas consultas será atualizado automaticamente para refletir essa mudança.

CREATE TABLE Consultas (

id INT AUTO_INCREMENT PRIMARY KEY,

beneficiario_id INT,

prestador_id INT,

data_consulta DATE,

procedimento_id INT,

valor_cobrado DECIMAL(10,2),

coparticipacao_pago DECIMAL(10,2),

FOREIGN KEY (beneficiario_id) REFERENCES Beneficiarios(id),

FOREIGN KEY (prestador_id) REFERENCES Prestadores(id),

FOREIGN KEY (procedimento_id) REFERENCES

Procedimentos(id));

CREATE TABLE Consultas (

id INT AUTO_INCREMENT PRIMARY KEY,

beneficiario_id INT,

prestador_id INT,

data_consulta DATE,

procedimento_id INT,

valor_cobrado DECIMAL(10,2),

coparticipacao_pago DECIMAL(10,2),

FOREIGN KEY (beneficiario_id) REFERENCES Beneficiarios(id)

ON DELETE CASCADE

ON UPDATE CASCADE,

FOREIGN KEY (prestador_id) REFERENCES Prestadores(id),

FOREIGN KEY (procedimento_id) REFERENCES Procedimentos(id));

Quando usar CASCADE com cuidado



Uso do DEFAULT

♦ DEFAULT **não é considerada uma constraint** pelo MySQL no sentido formal (como PRIMARY KEY, UNIQUE, FOREIGN KEY, CHECK).

♦ Apesar de DEFAULT impor um **comportamento automático de preenchimento**, o MySQL **não a registra como constraint** nas tabelas de metadados (INFORMATION_SCHEMA.TABLE_CONSTRAINTS). Ela é apenas **uma propriedade da coluna**.

Para definir o valor padrão do campo `status` como "Ativo", você pode usar a palavra-chave DEFAULT na definição da coluna.

```
CREATE TABLE Beneficiarios (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  nome VARCHAR(100),  
  data_nascimento DATE,  
  sexo ENUM('M', 'F', 'Outro'),  
  cpf CHAR(11) UNIQUE,  
  status ENUM('Ativo', 'Inativo', 'Cancelado'),  
  plano_id INT,  
  data_adesao DATE  
);
```

```
CREATE TABLE Beneficiarios (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  nome VARCHAR(100),  
  data_nascimento DATE,  
  sexo ENUM('M', 'F', 'Outro'),  
  cpf CHAR(11) UNIQUE,  
  status ENUM('Ativo', 'Inativo', 'Cancelado')DEFAULT 'Ativo',  
  plano_id INT,  
  data_adesao DATE  
);
```

Encerramento e revisão

Integridade de base

Constraints reforçam a integridade desde a fundação do banco.



Confiabilidade

Garantem que regras de negócio sejam sempre respeitadas.



Ferramentas disponíveis

MySQL oferece diversos mecanismos de proteção declarativa.



Aplicação crítica

Essenciais para sistemas sensíveis como o de saúde.

